



Orientação a Objetos

Diagramas de Casos de Uso (UML)

Prof. Edson José Pacheco

Fundamentos e Visão Geral

- ▶ O papel dos casos de uso na UML
- ▶ Visão funcional vs visão dinâmica
- ▶ Identificar atores e fronteiras do sistema
- ▶ Modelagem centrada no usuário

Relacionamentos

- ▶ Associação entre ator e caso de uso
- ▶ Relacionamento *include* (inclusão)
- ▶ Relacionamento *extend* (extensão)
- ▶ Generalização de atores e casos de uso

Elementos e Notação

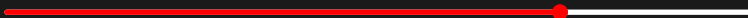
- ▶ Atores primários e secundários
- ▶ Casos de uso e associações
- ▶ Fronteira do sistema (*system boundary*)
- ▶ Notação visual padronizada UML

Prática e Mapeamento

- ▶ Traduzir requisitos em casos de uso
- ▶ Mapear casos de uso para código Python
- ▶ Exemplos em sistemas embarcados e IoT
- ▶ Exercício prático com automação industrial



CLIQUE PARA ABRIR NO NAVEGADOR



 Um Overview sobre UC

O Diagrama de Casos de Uso modela os **requisitos funcionais** do sistema, descrevendo o **que** o sistema faz do ponto de vista do usuário — sem entrar no *como*.

Características

- ▶ Faz parte da **visão funcional** da UML
- ▶ Perspectiva externa (caixa-preta)
- ▶ Descreve interações ator–sistema
- ▶ Independente de implementação
- ▶ Primeiro diagrama a ser construído no ciclo de desenvolvimento

Aplicações

- ▶ Levantamento e validação de requisitos
- ▶ Comunicação com stakeholders
- ▶ Planejamento de iterações e testes
- ▶ Definição do escopo e fronteira
- ▶ Base para estimativa de esforço

O diagrama responde à pergunta fundamental: *“O que o sistema precisa fazer para cada tipo de usuário?”*

Cenário: Sistema SCADA

Sem diagrama de casos de uso, a equipe começa a codificar a interface de supervisão sem definir claramente:

- ▶ Quem opera o sistema? (Operador? Engenheiro? Sistema externo?)
- ▶ Que funcionalidades cada perfil precisa?
- ▶ Onde termina o SCADA e começa o CLP?

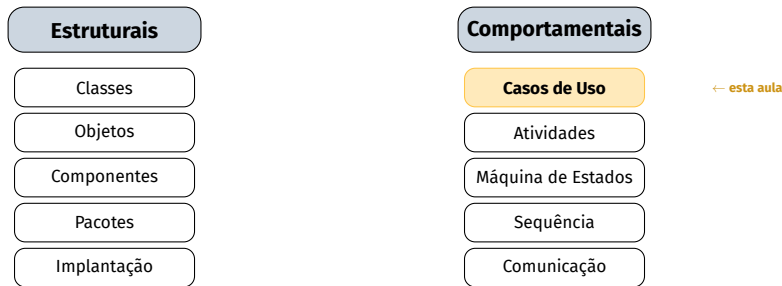
Com Casos de Uso

O diagrama esclarece **antes** da implementação:

- ▶ Operador: monitorar, acionar alarme
- ▶ Engenheiro: configurar setpoints, gerar relatórios
- ▶ CLP (ator externo): fornecer dados de processo
- ▶ Fronteira clara entre software e hardware

Em projetos de Engenharia Elétrica, a fronteira entre software e hardware é especialmente crítica.

A UML possui 14 tipos de diagramas, organizados em duas grandes categorias:



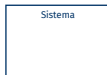
- ▶ Casos de Uso pertencem aos diagramas **comportamentais**
- ▶ É o diagrama mais **próximo do cliente/stakeholder**: linguagem não técnica
- ▶ Serve de **ponte** entre requisitos textuais e os demais diagramas UML

Elementos e Notação

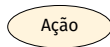




Ator



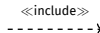
Fronteira



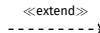
Caso de Uso



Associação



Inclusão



Extensão



Generalização

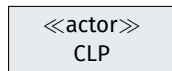
- ▶ **Ator:** Boneco palito — entidade externa que interage com o sistema
- ▶ **Caso de Uso:** Elipse — funcionalidade completa oferecida pelo sistema
- ▶ **Fronteira:** Retângulo — delimita o escopo do sistema
- ▶ **Relacionamentos:** Linhas e setas — conectam e qualificam os elementos

Atores: Entidade externa que interage com o sistema. Pode ser uma pessoa, outro sistema, um dispositivo ou até um **sensor**.

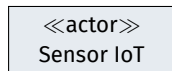
Tipos de Atores:

- ▶ **Primário:** Inicia a interação (convenção: lado esquerdo)
- ▶ **Secundário:** Participa, mas não inicia (convenção: lado direito)
- ▶ **Sistema externo:** Estereótipo «actor» em retângulo

Importante: Atores representam **papéis**, não indivíduos. Um engenheiro pode atuar como “Operador” e como “Administrador”.



(secundário)



(secundário)

Perguntas-chave para descobrir os atores do sistema:

Q Perguntas

- ▶ Quem usa o sistema diretamente?
- ▶ Quem fornece dados ao sistema?
- ▶ Quem recebe informações do sistema?
- ▶ Que hardware/dispositivo interage?
- ▶ Que sistemas externos se comunicam?
- ▶ Quem mantém/administra o sistema?

⚡ Ex: Monitoramento de Subestação

- ▶ **Operador** — monitora o painel
- ▶ **Eng. Proteção** — configura relés
- ▶ **Relé Digital** («actor») — envia dados de falta
- ▶ **SCADA Central** («actor») — coleta medições
- ▶ **Sistema de Alarme** («actor») — recebe notificações
- ▶ **Admin TI** — gerencia acessos

Em sistemas de EE, **dispositivos de campo** (CLPs, relés, sensores) frequentemente são atores secundários.

Caso de Uso: Funcionalidade completa que o sistema oferece a um ator. Sempre descrito com **verbo no infinitivo**.

- ▶ Representa um **objetivo** do ator
- ▶ Deve produzir um **resultado de valor**
- ▶ É independente de interface (UI/HMI)
- ▶ Nível adequado de granularidade
- ▶ Descreve *o que*, não *como*

Dica: Pergunte “O que o ator quer alcançar?”

Monitorar
tensão

✓ bom

Configurar
setpoint

✓ bom

Gerar relatório
de consumo

✓ bom

Processar
dados

✗ vago!

O **equivoco no nível de detalhe compromete a modelagem**. O caso de uso deve representar uma funcionalidade **completa com valor**, não um passo isolado.

Nível	Exemplo Ruim	Exemplo Bom
Muito fino	"Clicar botão Start"	"Iniciar processo de produção"
Muito fino	"Ler valor do sensor"	
Muito fino	"Exibir mensagem na HMI"	
Muito amplo	"Operar a planta inteira"	"Monitorar variáveis de processo"

Regra prática: Um caso de uso deve ser concluído em uma **sessão de interação** (minutos a horas). Se requer apenas poucos cliques, o nível é muito fino; se requer dias, é muito amplo.

O diagrama mostra a visão geral; a descrição textual detalha cada caso de uso individualmente.

Campo	Exemplo: Comandar Disjuntor
Nome do UC	Comandar equipamento
Ator primário	Operador
Pré-condições	Operador autenticado; disjuntor em estado estável
Pós-condições	Disjuntor no novo estado; evento registrado no Historian
Fluxo principal	1. Operador seleciona o disjuntor na IHM 2. Sistema exibe estado atual e solicita confirmação 3. Operador confirma o comando (abrir/fechar) 4. Sistema autentica operador (<<include>>) 5. Sistema envia comando ao IED via gateway 6. IED confirma execução 7. Sistema atualiza a tela e registra o evento
Fluxos alternativos	4a. Autenticação falha → exibir erro, encerrar 6a. IED não responde (timeout) → alarme de falha de comunicação
Regras de negócio	Intertravamento: não fechar se proteção ativa

Esta tabela é o **complemento essencial** do diagrama. O diagrama mostra o *mapa*; a descrição mostra o *roteiro*.

Cada caso de uso tem um fluxo principal e pode ter múltiplos fluxos alternativos:

Fluxo Principal

Caminho “feliz” (*happy path*):

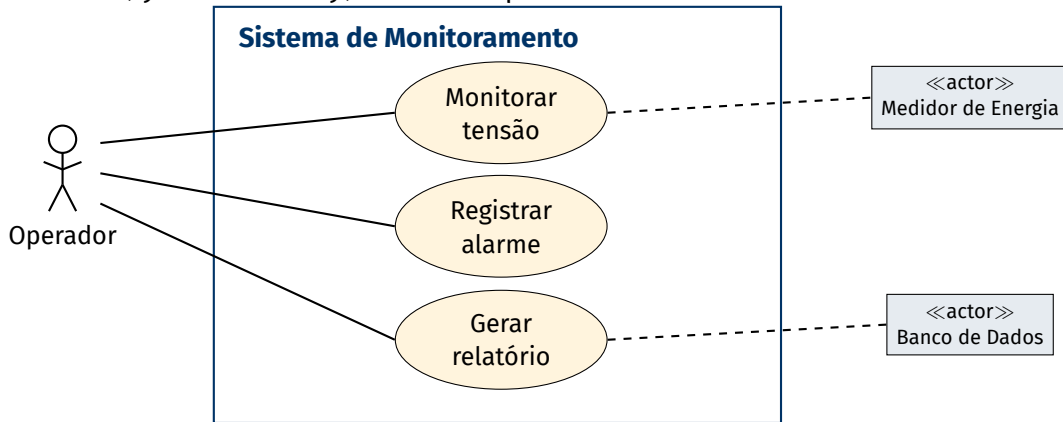
1. Operador seleciona disjuntor
2. Sistema valida e autentica
3. Comando enviado ao IED
4. IED confirma execução
5. Tela atualizada com sucesso

Fluxos Alternativos

- ▶ **2a.** Autenticação falha → bloquear após 3 tentativas
- ▶ **3a.** Intertravamento ativo → negar comando com justificativa
- ▶ **4a.** Timeout do IED → alarme de comunicação
- ▶ **4b.** IED rejeita (NACK) → registrar causa e notificar

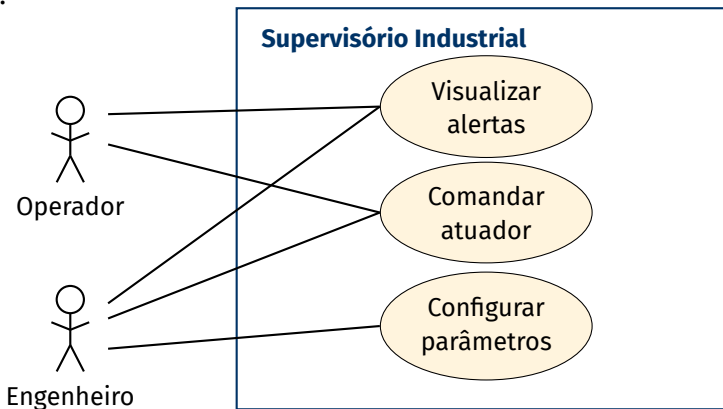
Regra prática: Para cada passo do fluxo principal, pergunte: “O que pode dar errado aqui?”
Em sistemas de EE, falhas de comunicação e intertravamentos são os cenários alternativos mais comuns.

A **fronteira** (*system boundary*) delimita o que está **dentro** e **fora** do sistema.



- ▶ Atores ficam **fora** — o medidor de energia é externo ao software
- ▶ Casos de uso ficam **dentro** — são funcionalidades do software
- ▶ A fronteira define o **escopo do que será implementado**

A **associação** conecta um ator a um caso de uso, indicando que o ator **participa** daquela funcionalidade.

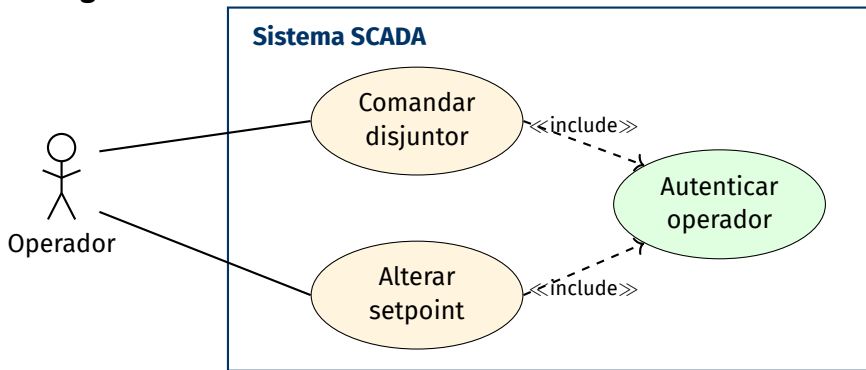


- ▶ Representada por **linha sólida** (sem seta obrigatória)
- ▶ **Operador** acessa 2 UCs; **Engenheiro** acessa 3 (incluindo configuração)
- ▶ Um ator pode se associar a múltiplos UCs e vice-versa

Relacionamentos



O **«include»** indica que um caso de uso **sempre** inclui o comportamento de outro. A execução é **obrigatória**.



- ▶ Seta tracejada: **caso base** → **caso incluído**
- ▶ “Comandar disjuntor” e “Alterar setpoint” **sempre** exigem autenticação
- ▶ **Motivação:** Fatorar comportamento **comum e obrigatório**

Use *include* quando:

✓ Situações corretas

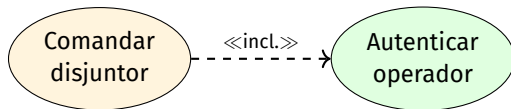
- ▶ Dois ou mais UCs **sempre** executam a mesma funcionalidade
- ▶ O comportamento extraído faz sentido **isoladamente**
- ▶ Autenticação, validação, logging são exemplos clássicos
- ▶ Leitura obrigatória de sensor antes de qualquer comando

✗ Armadilhas

- ▶ NÃO use para decompor um UC em passos sequenciais
- ▶ NÃO crie cadeias longas: A inclui B que inclui C...
- ▶ NÃO inclua algo que só um UC usa (sem reutilização)
- ▶ NÃO confunda com chamada de sub-rotina

Analogia EE: O *include* é como um regulador de tensão que **sempre** está presente antes de qualquer estágio do circuito.

O *include* mapeia para chamadas de método obrigatórias:



Python

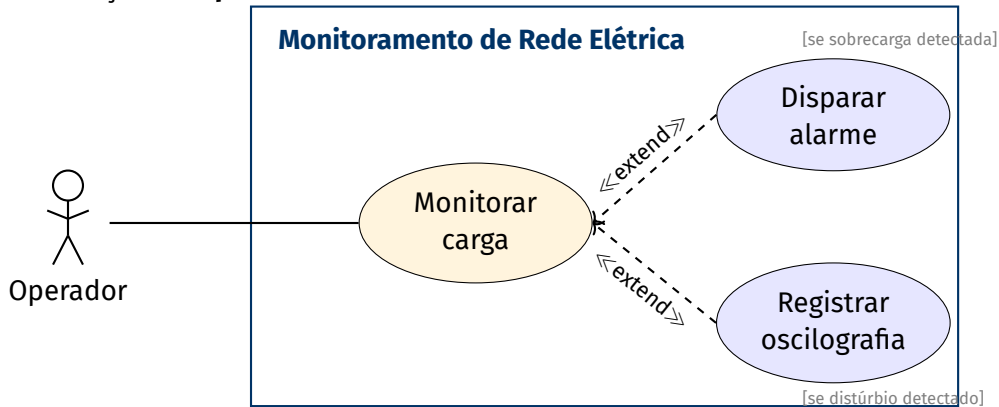
```
class SistemaSCADA:
    def autenticar_operador(self, user):
        """Caso de uso incluído (obrigatório)"""
        if not user.tem_permissao("operacao"):
            raise PermissionError("Sem permissao")
        self.log(f"Autenticado: {user.nome}")

    def comandar_disjuntor(self, user, id_disj,
                           acao: str):
        """Caso de uso base"""
        self.autenticar_operador(user) # <<include>>
        disj = Disjuntor.buscar(id_disj)
        disj.executar(acao) # "abrir" ou "fechar"

    def alterar_setpoint(self, user, tag, valor):
        """Outro caso de uso base"""
        self.autenticar_operador(user) # <<include>>
        variavel = Processo.get(tag)
        variavel.set_setpoint(valor)
```

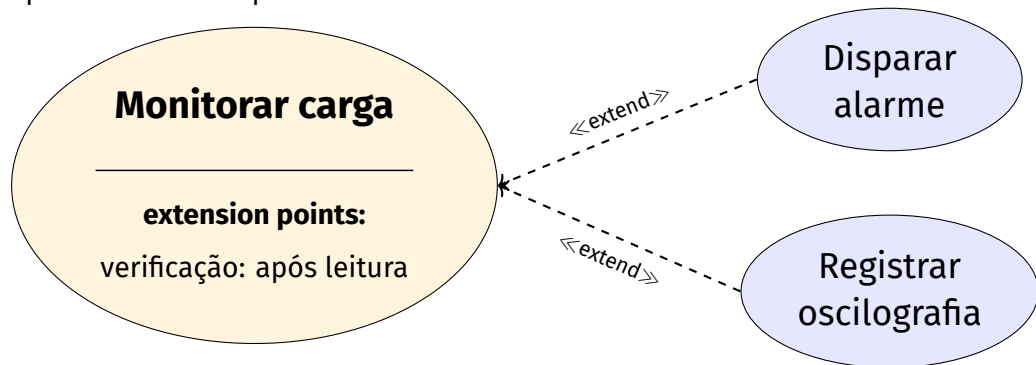
O método `autenticar_operador()` é chamado **sempre**, sem condicional — exatamente como o *include*.

O «**extend**» indica que um caso de uso **pode** adicionar comportamento a outro, sob certas condições. É **opcional**.



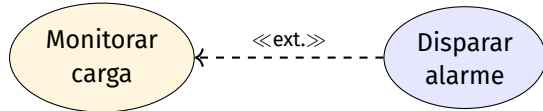
- ▶ Seta tracejada: **caso extensor** → **caso base** (atenção à direção!)
- ▶ “Disparar alarme” **só ocorre se** houver sobrecarga
- ▶ **Motivação:** Modelar funcionalidades **opcionais ou condicionais**

O caso base pode definir pontos de extensão (*extension points*), indicando **onde** o comportamento extra pode ser inserido.



- ▶ O ponto de extensão especifica o **momento exato** onde o extend pode ocorrer
- ▶ A condição de guarda (*[se sobrecarga]*) determina **se** ocorrerá
- ▶ O caso base **funciona normalmente** mesmo sem os extends

O **extend** mapeia para lógica condicional ou callbacks:



Python

```
class MonitoramentoRede:
    def disparar_alarme(self, medicao):
        """Caso de uso extensor"""
        alarme = Alarme(
            tipo="SOBRECARGA",
            valor=medicao.corrente,
            timestamp=datetime.now()
        )
        self.painel.notificar(alarme)

    def monitorar_carga(self, id_alimentador,
                        limiar=None):
        """Caso de uso base"""
        medicao = Sensor.ler(id_alimentador)
        self.registrar(medicao)

        # <<extend>> - so se sobrecarga
        if limiar and medicao.corrente > limiar:
            self.disparar_alarme(medicao)

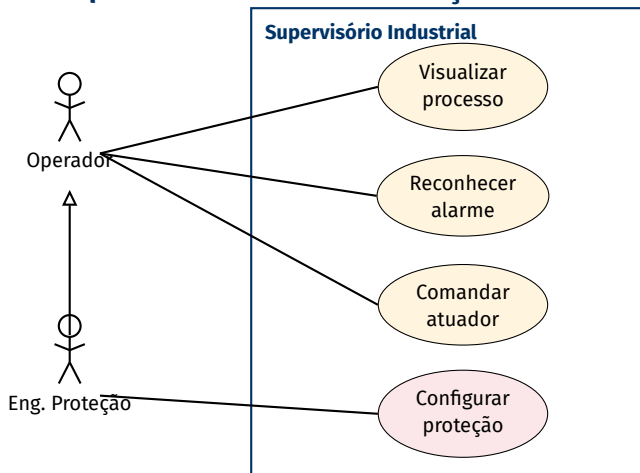
        return medicao
```

O `if limiar and medicao.corrente > limiar` expressa a **condicionalidade** do **extend**.

	«include»	«extend»
Execução	Sempre (obrigatório)	Condicional (opcional)
Direção	Base → Incluído	Extensor → Base
Quem sabe	O base sabe do incluído	O base NÃO sabe do extensor
Analogia EE	Regulador de tensão (sempre)	Proteção contra surtos (se necessário)
Em Python	<code>self.metodo()</code> direto	<code>if cond: self.metodo()</code>
Propósito	Reutilizar comportamento	Adicionar funcionalidade

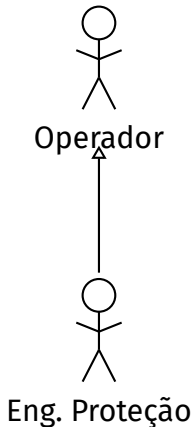
Macete: “*include* = sempre executa” e “*extend* = às vezes executa”. A seta do *extend* aponta na direção **oposta** à do *include*!

A generalização permite que um ator herde as associações de um ator mais geral.



- ▶ Eng. Proteção **herda** as funcionalidades do Operador.
- ▶ Eng. Proteção possui a funcionalidade exclusiva de “Configurar proteção”.
- ▶ A seta com **triângulo aberto** indica a direção da herança (aponta para o pai).

A generalização de atores mapeia diretamente para herança de classes:



Python

```
class Operador:
    """Ator geral"""
    def visualizar_processo(self, id_proc):
        return Processo.get(id_proc).dados

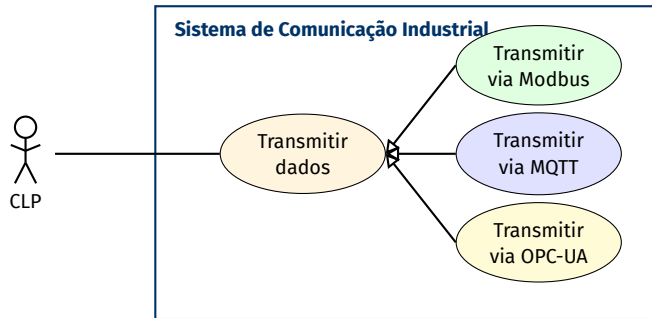
    def reconhecer_alarme(self, id_alarme):
        Alarme.get(id_alarme).reconhecer()

    def comandar_atuador(self, tag, valor):
        Atuador.get(tag).comando(valor)

class EngenheiroProtecao(Operador):
    """Herda tudo do Operador"""
    def configurar_protecao(self, id_rele,
                           params: dict):
        rele = ReleDigital.get(id_rele)
        rele.atualizar_ajustes(params)
```

EngenheiroProtecao herda `visualizar_processo()`, `reconhecer_alarme()` e `comandar_atuador()` automaticamente.

Casos de uso também suportam **generalização**, representando **variações** de uma mesma funcionalidade.



- ▶ “Transmitir via Modbus”, “MQTT” e “OPC-UA” são **especializações** de “Transmitir dados”
- ▶ Cada uma herda o comportamento base e adiciona seu protocolo específico
- ▶ O ator interage com o caso geral; o sistema escolhe a especialização

Especializações mapeiam para classes abstratas com polimorfismo:

Python

```
from abc import ABC, abstractmethod

class Transmissor(ABC):
    """Caso de uso geral (abstrato)"""
    @abstractmethod
    def transmitir(self, dados: bytes) -> bool:
        pass

class TransmissorModbus(Transmissor):
    def transmitir(self, dados: bytes) -> bool:
        return self.porta_serial.write(
            self.montar_frame_rtu(dados))

class TransmissorMQTT(Transmissor):
    def transmitir(self, dados: bytes) -> bool:
        return self.client.publish(self.topico, dados)

class TransmissorOPCUA(Transmissor):
    def transmitir(self, dados: bytes) -> bool:
        return self.node.set_value(ua.Variant(dados))
```

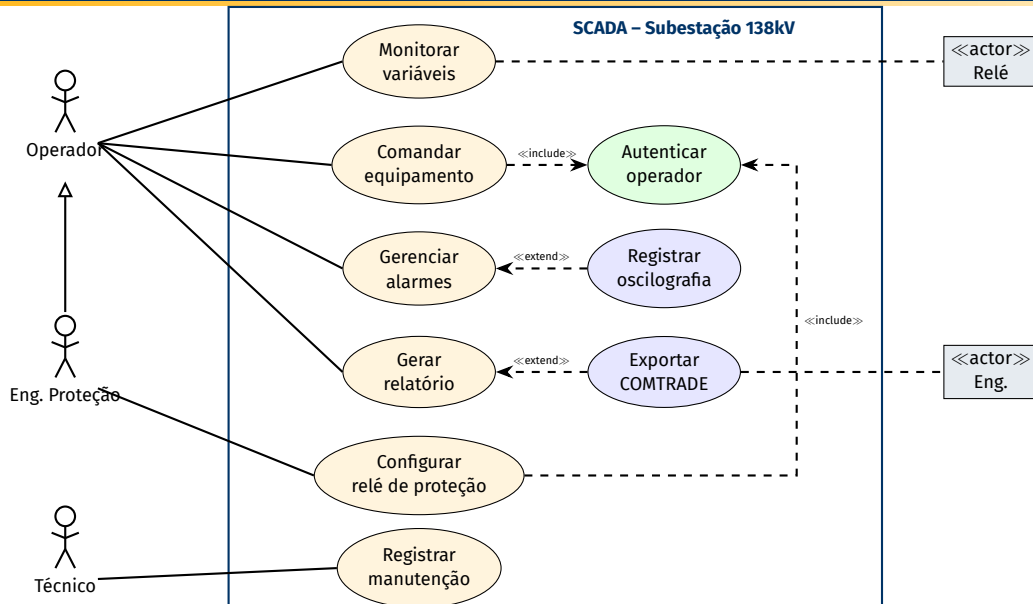
O polimorfismo permite trocar o protocolo sem alterar o código chamador:

```
transmissor.transmitir(dados)
```

Exemplos Completos



Exemplo 1: Sistema SCADA de Subestação



```
class SCADASubestacao:
    def autenticar_operador(self, user):          # <<include>>
        if not user.tem_nivel("operacao"):
            raise PermissionError("Nível insuficiente")

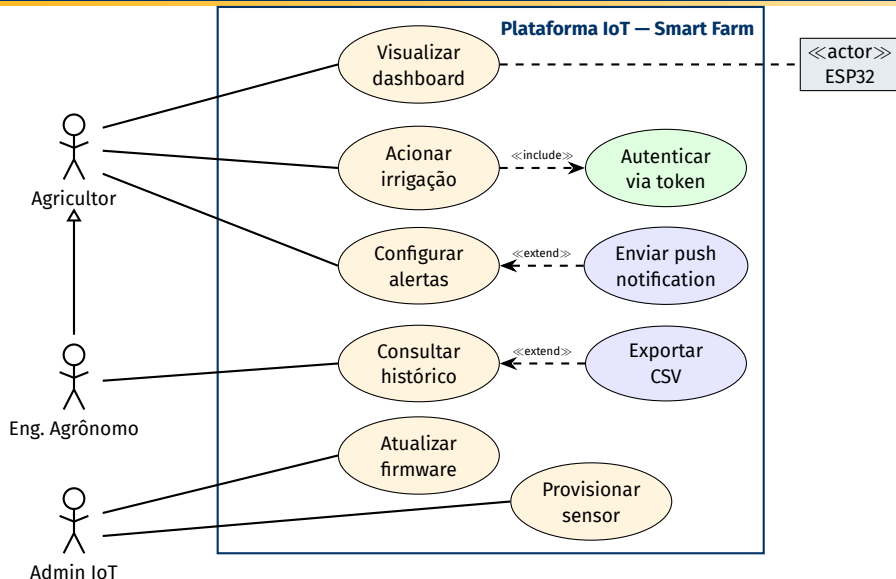
    def monitorar_variaveis(self, id_bay):        # UC principal
        leituras = self.ied_client.read_all(id_bay) # Comunica com IED (ator)
        return {tag: val for tag, val in leituras.items()}

    def comandar Equipamento(self, user, tag, cmd): # UC com <<include>>
        self.autenticar_operador(user)           # <<include>> obrigatorio
        return self.ied_client.send_command(tag, cmd)

    def gerenciar_alarmes(self, filtros,
                          registrar_oscilo=False):
        alarmes = Alarme.query.filter_by(**filtros).all()
        if registrar_oscilo:                       # <<extend>> oscilografia
            for a in alarmes:
                self._registrar_oscilografia(a)
        return alarmes

    def gerar_relatorio(self, periodo,
                       exportar_comtrade=False):
        dados = self.historian.consultar(periodo)  # Comunica com Historian
        if exportar_comtrade:                      # <<extend>> COMTRADE
            return self._exportar_comtrade(dados)
        return RelatorioHTML(dados)
```

Exemplo 2: IoT – Agricultura de Precisão




```
import paho.mqtt.client as mqtt
class Agricultor:
    """Ator primario"""
    def visualizar_dashboard(self):
        return SensorNode.get_latest_readings()    # Dados do ESP32

    def acionar_irrigacao(self, zona: int,
                          duracao_min: int):
        self._autenticar_via_token()                # <<include>>
        payload = {"zona": zona, "duracao": duracao_min}
        self.mqtt_client.publish(
            f"farm/irrigacao/{zona}",
            json.dumps(payload))

    def configurar_alertas(self, regras, push=False):
        Alerta.salvar_regras(regras)
        if push:                                     # <<extend>> notificacao
            PushService.registrar(self.device_token)

class EngenheiroAgronomo(Agricultor):
    """Herda dashboard, irrigacao e alertas"""
    def consultar_historico(self, periodo,
                           exportar_csv=False):
        dados = TimeSeries.query(periodo)
        if exportar_csv:                             # <<extend>> CSV
            return dados.to_csv()
        return dados
```

Exercício Prático



Desafio: Crie um diagrama de casos de uso para o seguinte cenário:

Sistema de Controle de Linha de Produção

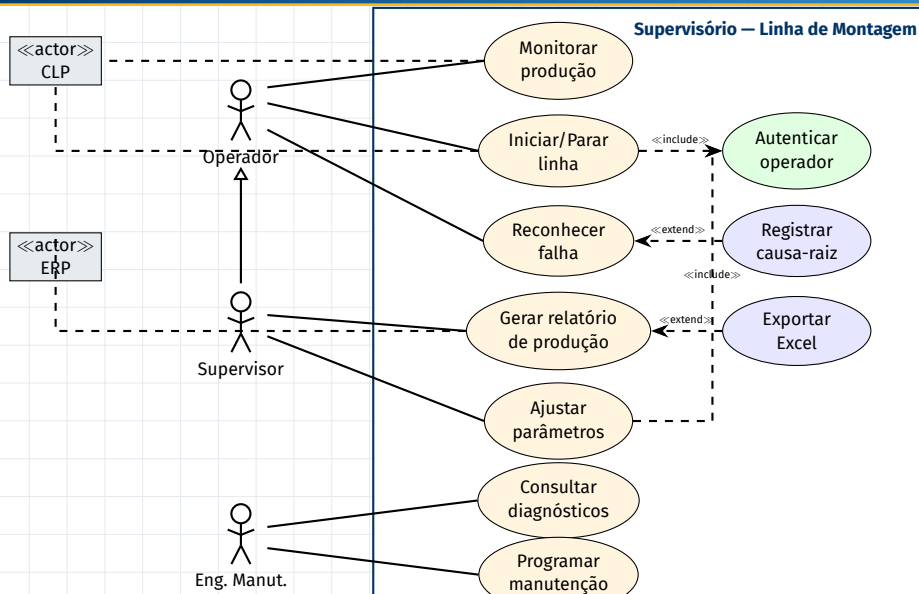
Uma fábrica possui um sistema supervisorio para sua linha de montagem:

1. **Operador:** pode monitorar produção, iniciar/parar linha e reconhecer falhas
2. **Supervisor** (herda de Operador): pode gerar relatórios e ajustar parâmetros
3. **Eng. Manutenção:** pode consultar diagnósticos e programar manutenção
4. Iniciar linha e ajustar parâmetros **sempre** requerem autenticação (*include*)
5. Gerar relatório **pode** exportar para Excel (*extend*)
6. Reconhecer falha **pode** registrar causa-raiz (*extend*)
7. Atores externos: **CLP** (fornece dados) e **ERP** (recebe indicadores)

Identifique: 3 atores primários + 2 secundários, generalização, 8+ UCs, includes e extends



Solução: Controle de Linha de Produção



Elementos identificados no diagrama:

Atores e Herança:

- ▶ **Operador:** Monitorar, iniciar/parar, reconhecer falhas
- ▶ **Supervisor:** Herda do Operador + relatórios e ajustes
- ▶ **Eng. Manut.:** Diagnósticos e manutenção (papel independente)
- ▶ **CLP / ERP:** Atores secundários externos

Relacionamentos:

- ▶ **Include:** Autenticação para iniciar e ajustar
- ▶ **Extend:** Exportar Excel e registrar causa-raiz
- ▶ **Generalização:** Supervisor herda 3 UCs do Operador
- ▶ **Eng. Manut.:** NÃO herda — papel distinto

Ponto-chave: Eng. Manutenção NÃO é generalização do Operador — são papéis com responsabilidades diferentes. Generalização só faz sentido quando há **extensão de responsabilidades**.

Concluindo...



Elemento UML	Mapeamento Python	Exemplo EE
Ator	Classe ou role	<code>class Operador</code>
Caso de Uso	Método público	<code>def comandar()</code>
«include»	Chamada obrigatória	<code>self.autenticar()</code>
«extend»	Condicional / if	<code>if alarme: oscilo()</code>
Generalização (ator)	Herança de classe	<code>class Eng(Op)</code>
Generalização (UC)	Polimorfismo / ABC	<code>class Transmissor</code>
Fronteira do sistema	Módulo / Package	<code>scada.py</code>
Ator secundário	Dependência externa	<code>paho.mqtt</code>

Casos de Uso	Classes	Atividades
O que o sistema faz	Como é estruturado	Como o processo flui
Visão funcional	Visão estrutural	Visão dinâmica
Foco no usuário	Foco nos dados	Foco no fluxo
Requisitos	Projeto detalhado	Algoritmo / Workflow
Caixa-preta	Caixa-branca	Caixa-branca

- ▶ Use **Casos de Uso** para capturar *o que* o sistema faz para cada ator
- ▶ Use **Classes** para detalhar a estrutura interna (atributos, métodos)
- ▶ Use **Atividades** para detalhar o fluxo de cada caso de uso

Ao criar Diagramas de Casos de Uso:

FAÇA:

- ▶ Use verbos no infinitivo (“Monitorar tensão”)
- ▶ Mantenha entre 5–15 UCs por diagrama
- ▶ Primários à esquerda, secundários à direita
- ▶ Valide com operadores e engenheiros
- ▶ Use *include* para comportamento comum
- ▶ Dispositivos de campo como atores secundários

EVITE:

- ▶ UCs que são passos (“Clicar botão da HMI”)
- ▶ Mais de 3 níveis de *include/extend*
- ▶ Direção errada da seta de *extend*
- ▶ Modelar a interface (telas da HMI)
- ▶ Detalhar demais (papel de atividades)
- ▶ CRUD como 4 UCs (use “Gerenciar X”)

Armadilhas frequentes ao modelar casos de uso:

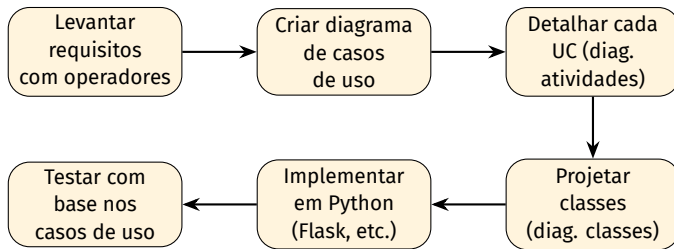
✘ Erro

- ▶ UC “Ler registrador Modbus” (passo técnico, não UC)
- ▶ Seta do *extend* apontando do base para o extensor
- ▶ “Inserir sensor”, “Editar sensor”, “Remover sensor” como 3 UCs
- ▶ CLP desenhado **dentro** da fronteira do sistema
- ▶ Usar *include* para algo que só um UC usa

✔ Correto

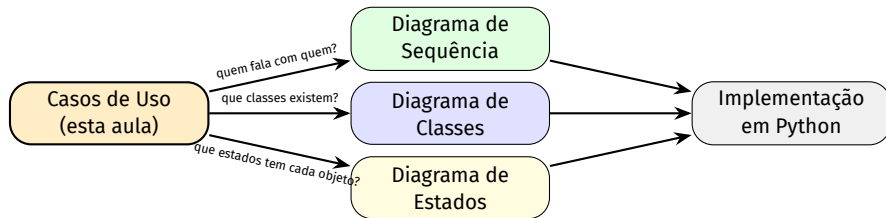
- ▶ UC “Monitorar variáveis de processo” (funcionalidade completa)
- ▶ Seta do *extend*: extensor → base
- ▶ “Gerenciar sensores” (engloba CRUD)
- ▶ CLP como «actor» **fora** da fronteira
- ▶ *Include* quando 2+ UCs compartilham comportamento

Fluxo de trabalho recomendado para projetos de EE:



Dica: Cada caso de uso gera um ou mais **testes** em pytest. Os *includes* viram fixtures obrigatórias e os *extends* viram testes parametrizados (com e sem a extensão).

O diagrama de casos de uso é o ponto de partida. Cada UC será detalhado nos próximos diagramas:



Próxima aula: Diagrama de Sequência

- ▶ Detalha a ordem das mensagens entre objetos
- ▶ Revela os métodos necessários nas classes
- ▶ Modela protocolos: Modbus, MQTT, IEC 61850

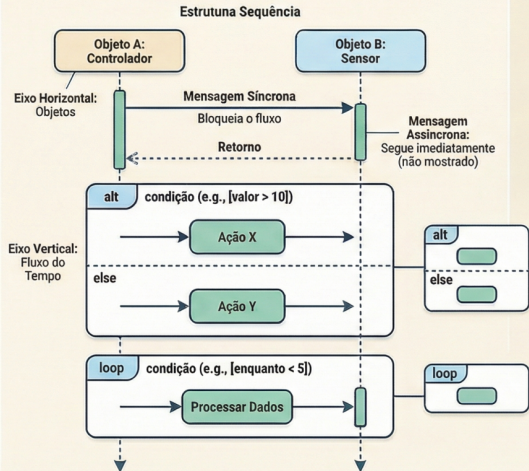
UC "Comandar equipamento" gera:

- ▶ 1 sequência: cenário principal
- ▶ 1 sequência: timeout do IED
- ▶ 1 sequência: falha de autenticação

Dominando Diagramas de Sequência: Da UML ao Código Python

O Diagrama de Sequência modela a interação dinâmica entre objetos ao longo do tempo. Este guia foca na notação visual e na tradução direta para estruturas de código Python.

Fundamentos e Notação Visual

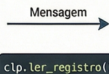


Mapeamento UML para Python



Data Table

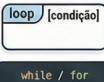
Mensagem Síncrona



Fragmento alt



Fragmento loop





Prof. Edson José Pacheco

pacheco@ufpr.br

UFPR

UNIVERSIDADE FEDERAL DO PARANÁ